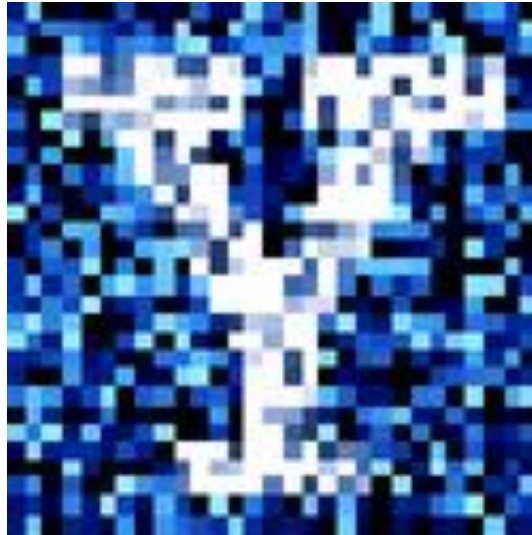


YData: Introduction to Data Science



Class 13: Maps



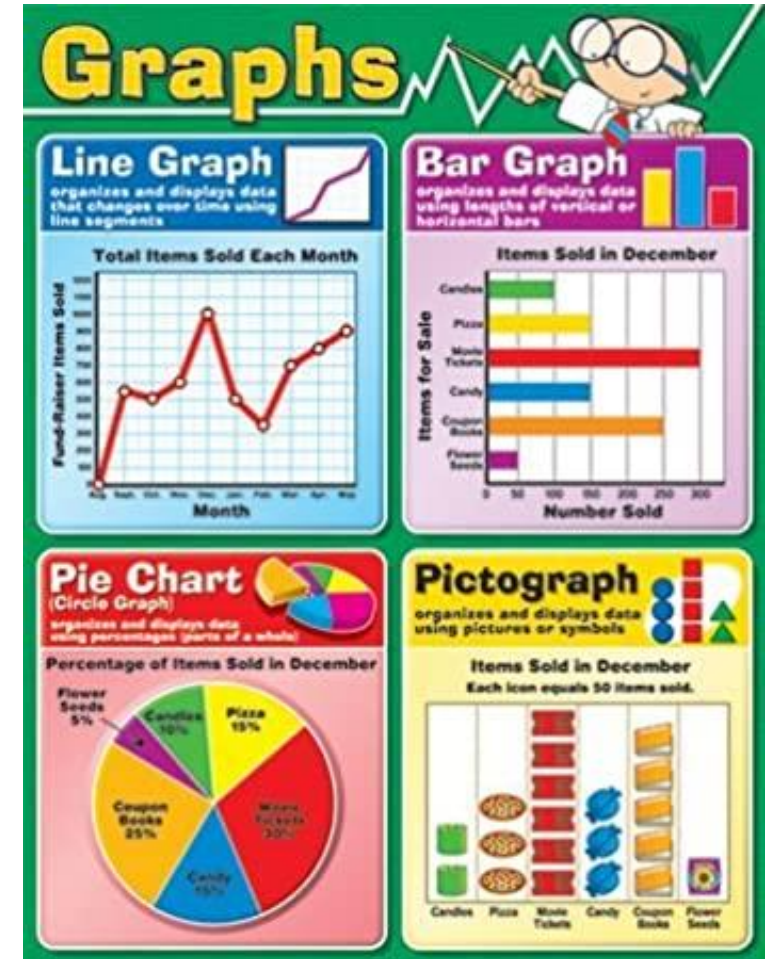
I just collected a table of snow data!

Overview

Pivot Table and Heatmaps

Maps

- Brief history of visualizing data with maps
- Geopandas
- Coordinate reference systems and projections
- Choropleth maps



Announcement

- The midterm is postponed to Thursday, in-person.
 - A few of you with special accommodation, please note that we have synced with SAS the new time.
- HW 5 is due on Gradescope on **Sunday Mar. 1st at 11pm**
- We will release HW 6 on Thursday, which will be due on **Sunday Mar. 8th at 11pm.**
 - This will be a short homework covering only today's lecture.

Announcement: class project

The final project is a Jupyter notebook report where you analyze your own data to address a question that you find interesting

- It's a chance to practice everything you've learned in class!

The goal of is to present a clear and compelling data analysis showing a few interesting results!

- Ideally something you are proud of that you could show off to potential future employers, etc.

A few sources for data sets will be listed on Canvas

- You can use data you collect as well. If you use data from another class, your work must be unique for each class

Announcement: class project

A draft of the project is due on **April 5th**

- I'm telling you about this early so:
 - If you want to think/work on the project over break you can
 - If don't want to think about it over break you don't have to

There will be a “peer review” period where you will give and receive feedback on three other projects

- Instructions for how to do the review will be given

The final version will be due on **April 23rd**

A Jupyter notebook “template” project will be available soon

Questions?

Pivot Tables and heatmaps

Pivot Tables and heatmaps

Pivot tables aggregate values based on two grouping variables, and create a table where:

- The rows are the levels of one cat. variable
- The columns are the levels of the second cat. variable
- The values are aggregated over a third quant. variable

```
df2 = df.pivot_table(index = "col1", columns = "col2",  
                      values = "col3", aggfunc = "max")
```

```
nba2 = nba.pivot_table(index = "TEAM", columns = "POSITION",  
                       values = "SALARY", aggfunc = "max")
```

rows columns values

	PLAYER	TEAM	POSITION	SALARY
0	De'Andre Hunter	Atlanta Hawks	SF	9.835881
1	Jalen Johnson	Atlanta Hawks	SF	2.792640
2	AJ Griffin	Atlanta Hawks	SF	3.536160
3	Trent Forrest	Atlanta Hawks	SG	0.508891
4	John Collins	Atlanta Hawks	PF	23.500000

	POSITION	C	PF	PG	SF	SG
TEAM						
Atlanta Hawks		18.206896	23.500000	37.096500	9.835881	17.071120
Boston Celtics		26.500000	4.306281	22.800000	30.351780	17.142857
Brooklyn Nets		9.391069	44.119845	38.917057	20.100000	19.500000
Charlotte Bobcats		3.722040	1.563518	8.623920	30.075000	21.486316
Chicago Bulls		3.200000	7.775400	9.030000	27.300000	37.096500

Pivot Tables and heatmaps

One can then visualize the data as a heatmap using seaborn

```
sns.heatmap(nba2) # seaborn
```

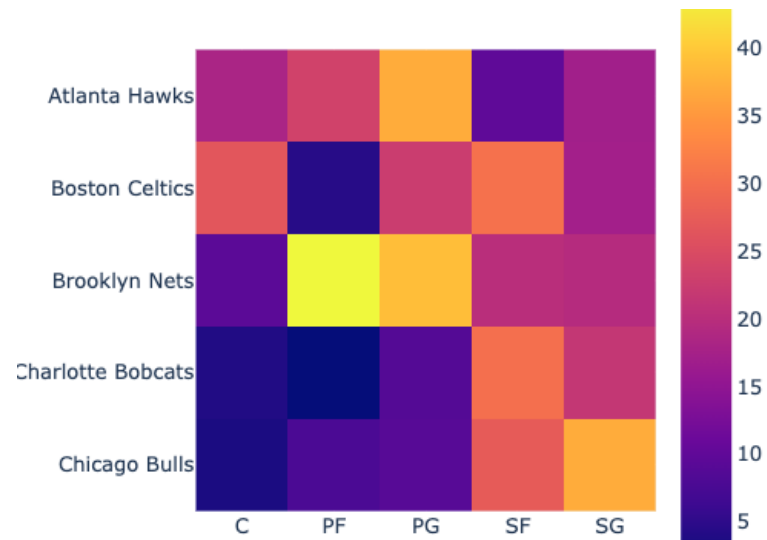


Diagram illustrating the relationship between a data table and its pivot table:

The original data table (left) has columns: **PLAYER**, **TEAM**, **POSITION**, and **SALARY**. The **TEAM**, **POSITION**, and **SALARY** columns are highlighted with red boxes and labeled "rows", "columns", and "values" respectively.

	PLAYER	TEAM	POSITION	SALARY
0	De'Andre Hunter	Atlanta Hawks	SF	9.835881
1	Jalen Johnson	Atlanta Hawks	SF	2.792640
2	AJ Griffin	Atlanta Hawks	SF	3.536160
3	Trent Forrest	Atlanta Hawks	SG	0.508891
4	John Collins	Atlanta Hawks	PF	23.500000

Red arrows indicate the transformation to the pivot table (right), where the **POSITION** column becomes the index, and the **TEAM** and **SALARY** columns are pivoted into columns.

POSITION	C	PF	PG	SF	SG
TEAM					
Atlanta Hawks	18.206896	23.500000	37.096500	9.835881	17.071120
Boston Celtics	26.500000	4.306281	22.600000	30.351780	17.142857
Brooklyn Nets	9.391069	44.119845	38.917057	20.100000	19.500000
Charlotte Bobcats	3.722040	1.563518	8.623920	30.075000	21.486316
Chicago Bulls	3.200000	7.775400	9.030000	27.300000	37.096500

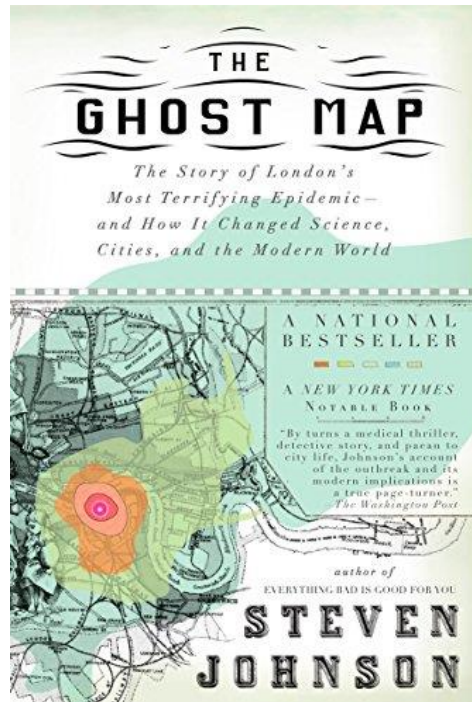
Let's explore this in Jupyter!

Maps

Maps to determine the causes of cholera

Visualizing data on a map can be a powerful way to see spatial trends

- One of the first maps used to show spatial trends was created by John Snow to further his case that cholera was a water born illness



Cholera in London in the 19th century

Cholera reached London in early 1830s

It was greatly feared as it was often deadly

- An outbreak in 1849 killed over 14,000 people in London

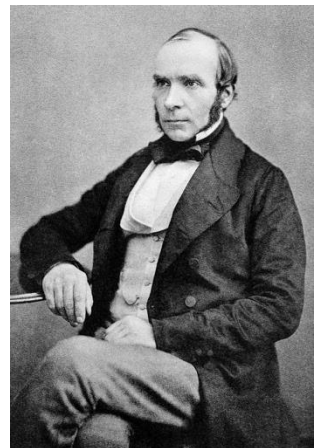
Cause of cholera was unknown. Several theories:

1. Miasmas theory: caused by bad air/smells

- Florence Nightingale, Edwin Chadwick (board of health)

2. Water born disease

- John Snow (anesthesiologist)

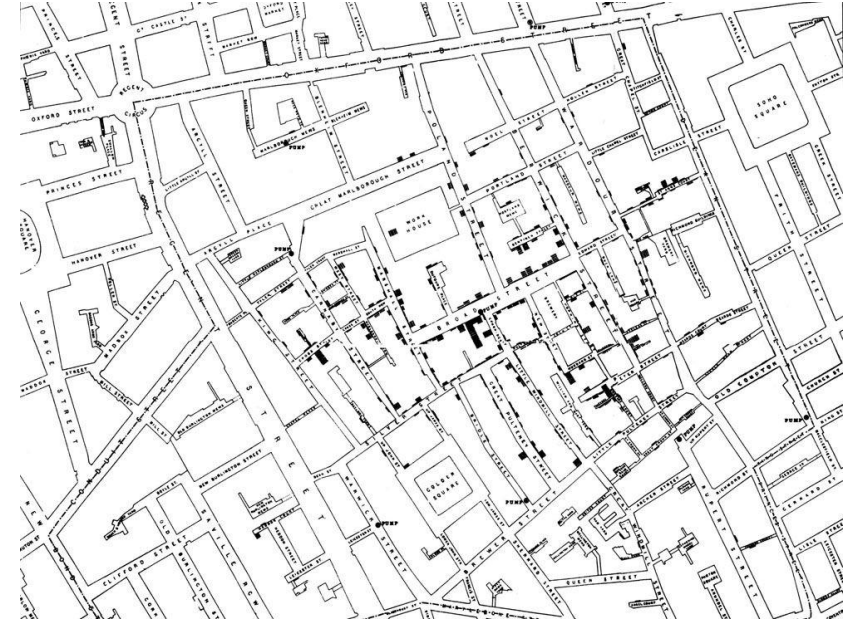


John Snow and spatial mapping

To try to understand the cause of the cholera outbreak of 1854, John Snow plotted a map of cholera deaths

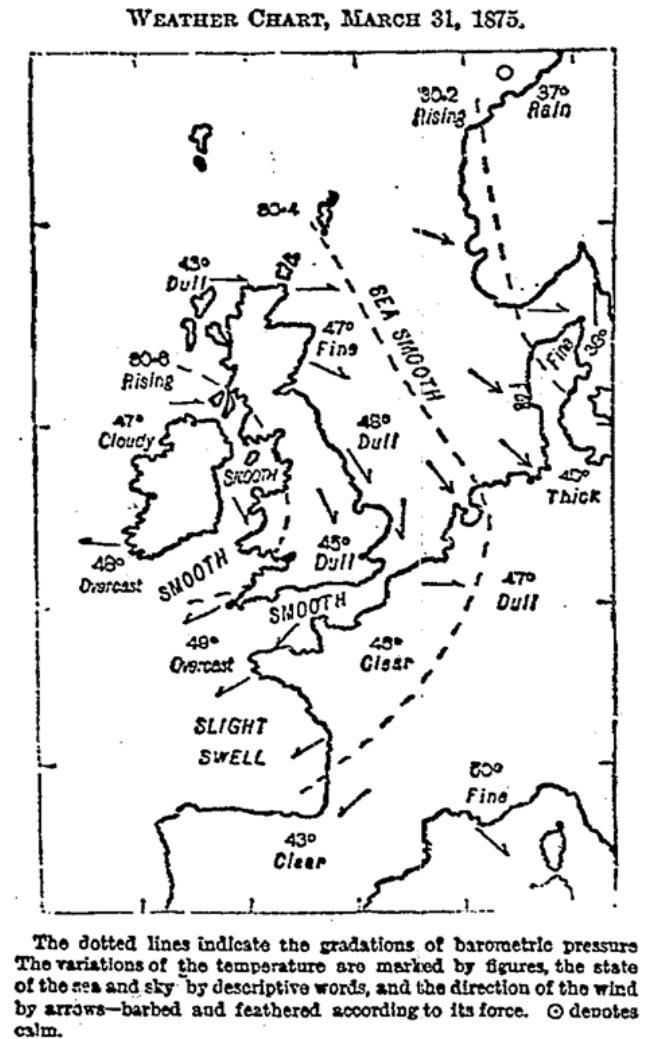
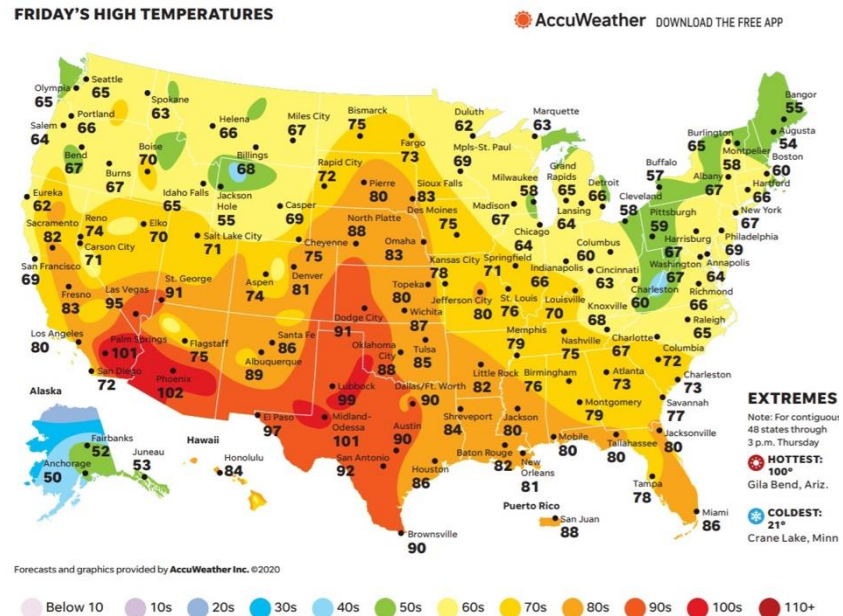
Based on this map and interviews, he concluded that the source of cholera was the Broad Street well

- He famously removed the handle of the well to prevent the spread of disease
- Now he is considered the founder of epidemiology



Maps

Another early use where a map gave insight was the mapping of weather by John Galton in 1875



Galton's first weather map (1875)

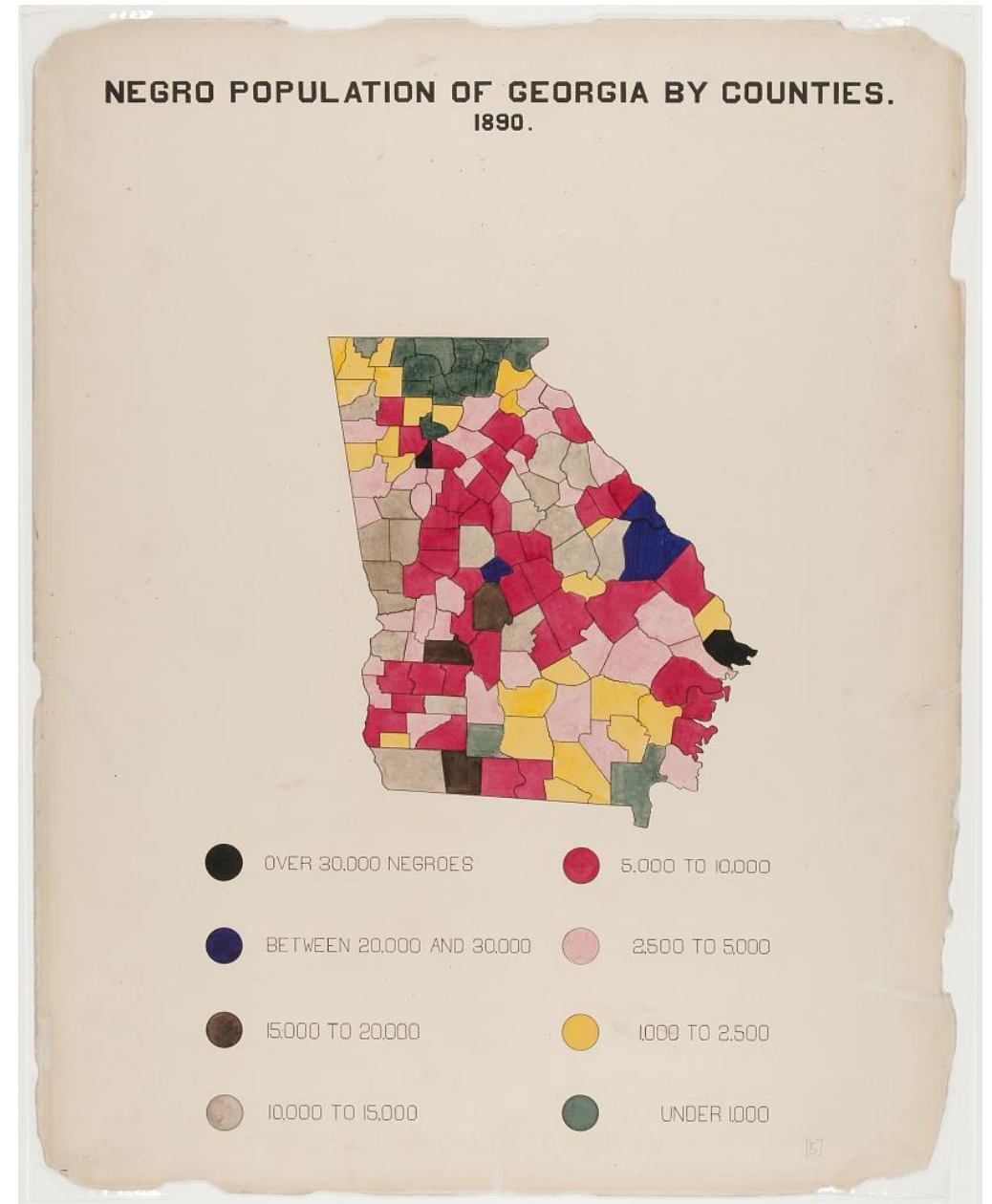
W. E. B. Du Bois

W.E.B. Du Bois was a social scientist and prominent African-American rights activist

Took on the complex task of gathering and manually visualizing the lives of Black Americans in the 1890s

Presented 58 visualizations in the 1900 World's Fair in Paris

- Won a gold award



geopandas

To create maps in Python we will use the geopandas package

```
import geopandas as gpd
```

The key object of interest is the geopandas DataFrame

- It is the same as a regular data frame but it has an extra column called "geometry" that contains geospatial shape features
- The geometry column contains "Shapely" objects used to represent geometric shapes

	key_comb_drvr	geometry
0	M11551	POINT (117.525391 34.008926)
1	M17307	POINT (86.51248 30.474344)
2	M19584	POINT (89.537415 37.157627)
3	M21761	POINT (117.526871 34.00647)
4	M22374	POINT (117.525345 34.008915)
5	U01997A	POINT (84.80533 33.719654)
6	U153601	POINT (78.24838 39.986454)
7	U159393	POINT (98.49438499999999 40.801544)
8	U722222	POINT (84.23309 33.9386)
9	U723030	POINT (83.86456 34.08479)
10	U723333	POINT (85.67151 42.83093)
11	U753333	POINT (117.498535 34.069157)
12	U760505	POINT (90.61252 41.456993)

geopandas

We can read in data as a geopandas DataFrame using

```
map = gpd.read_file('my_file.geojson')
```

We can plot maps using the `gpd.plot()` function

Let's explore this in Jupyter!

Coordinate reference systems

A coordinate reference system (CRS) is a framework used to precisely measure locations on the surface of the Earth as coordinates

The goal of any coordinate reference system is to create a common reference frame in which locations can be measured precisely as coordinates, so that any recipient can identify the same location that was originally intended

- Needed for aligning different layers on maps

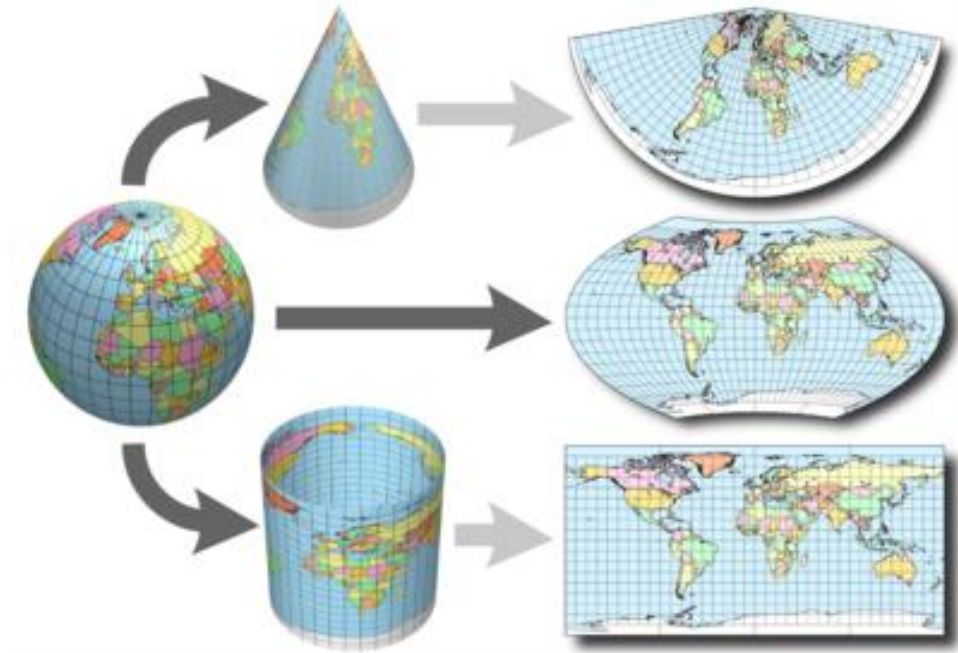


Map projections

Since the earth is a 3D structure, coordinate systems have to project their data onto a 2D maps

Different projects preserve different properties

- **Mercator projection** keeps angles intact
 - Useful for navigation
- **Eckert IV projection** keeps the size of land areas intact



Let's explore this in Jupyter!

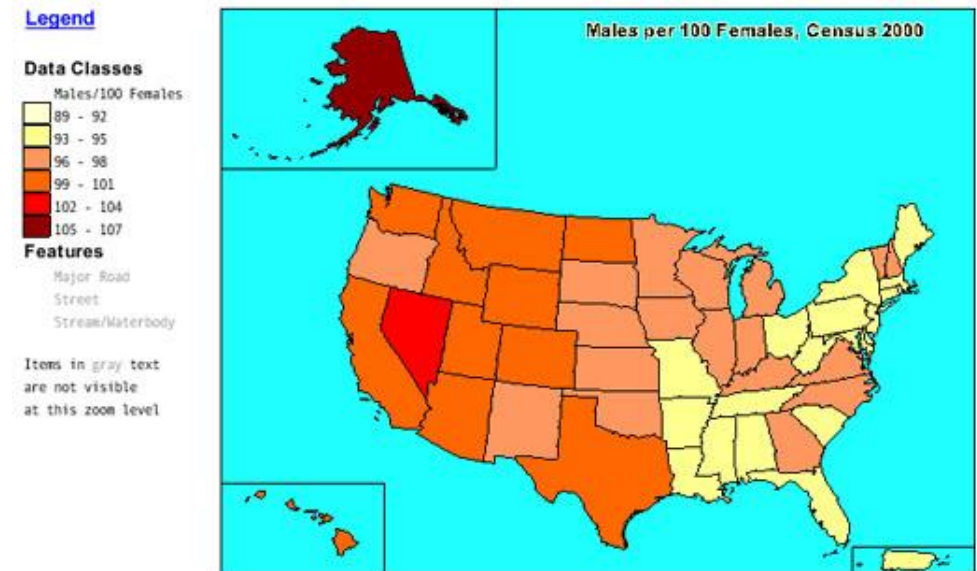
Choropleth maps

Choropleth maps: shades/colors in predefined areas based on properties of a variable

We can create choropleth maps using geopandas by joining region information on to a geopandas DataFrame that has a map

We can then use the `gpd.plot(column = )` method to visualize the map

Choropleth map



Let's explore this in Jupyter!